

Decision Tree, Ensemble Learning & Hyperparameter Tuning

PTP Program





Syarat dan Aturan dalam Pelatihan

Untuk mengikuti materi pelatihan berikut ada beberapa hal yang harus diperhatikan oleh peserta:

- Peserta diharapkan sudah memiliki pengetahuan dasar tentang Data Analysis, Machine Learning dan Feature Engineering .
- Peserta diharapkan memperhatikan materi dan mencatat materi atau penjelasan yang disampaikan dari instruktur.
- Instruktur diharapkan dapat menjelaskan materi dengan jelas dan memastikan peserta pelatihan dapat memahami topik yang dibahas.
- Materi yang diajarkan sebagian besar adalah praktik atau menggunakan Google Colab (Jupyter Notebook).



Daftar Isi

- Decision Tree
- Ensemble Learning
- Hyperparameter Tuning
- Hands-On



Objectives

- Peserta dapat mengetahui tentang Decision Tree dan cara penerapannya.
- Peserta dapat mengetahui tentang Ensemble Learning dan cara penerapannya.
- Peserta dapat memahami cara melakukan Hyperparameter tuning.



Decision Tree

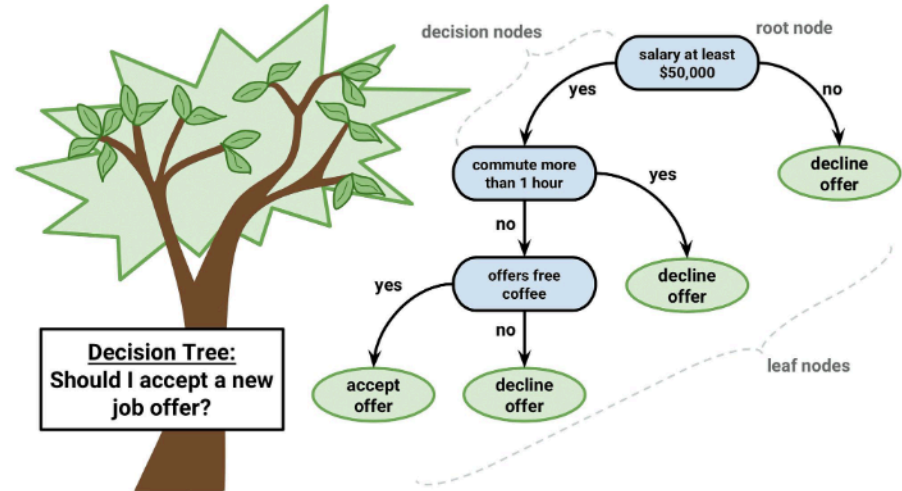
Decision Tree

Apa itu Decision Tree?

Decision Tree adalah salah satu algoritma **Supervised Learning** yang dapat digunakan untuk **klasifikasi** dan **regresi**.

Tujuan utamanya adalah membuat model yang bisa memprediksi nilai target dengan mempelajari aturan keputusan sederhana dari fitur-fitur data.

Decision Tree disebut sebagai algoritma **non-parametrik**, artinya tidak mengasumsikan bentuk tertentu dari distribusi data, berbeda dengan **Linear Regression** yang mengasumsikan hubungan linear.



Sumber gambar: www.medium.com

Decision Tree

Struktur Decision Tree

Ada 3 jenis node utama yang membentuk struktur Decision Tree:

1. **Root Node (Node Akar)**
 - Node ini berada di bagian paling atas dari Decision Tree.
 - Menjadi titik awal proses pengambilan keputusan.
 - Atribut yang paling penting ditempatkan di root node.
2. **Decision Node (Node Keputusan/Node Internal)**
 - Node ini adalah titik di tree tempat keputusan lanjutan dibuat berdasarkan nilai dari suatu fitur.
 - Setiap node berisi fitur dan aturan keputusan yang membagi data menjadi beberapa cabang atau subset.
3. **Leaf Node (Node Daun)**
 - Node ini mewakili hasil akhir atau prediksi dari model.
 - Tidak ada percabangan lebih lanjut dari node ini.

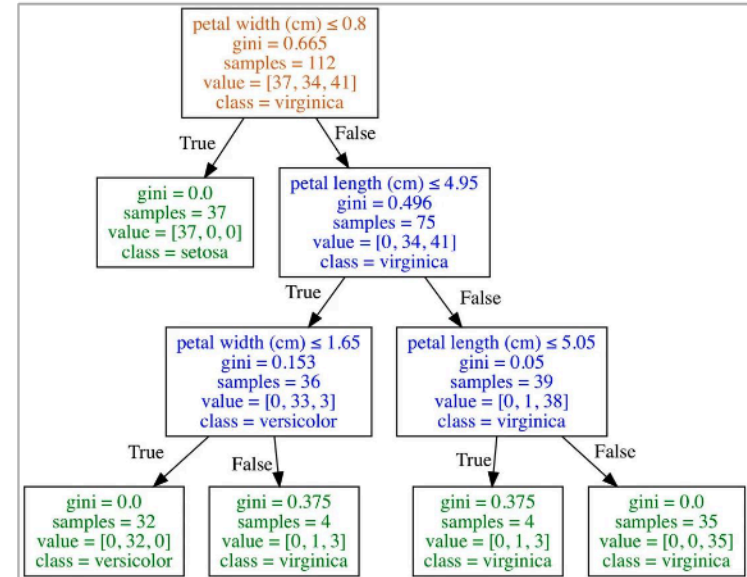


Illustration of a Decision Tree in Iris dataset
 Orange-text : Root node
 Blue-text : Decision Nodes (Internal Nodes)
 Green-text : Leaf Nodes

Decision Tree

Kelebihan Dan Kekurangan

Kelebihan Decision Tree

1. **Mudah dipahami dan diinterpretasi**, serta dapat divisualisasikan.
2. **Metode non-parametrik**, artinya tidak bergantung pada asumsi distribusi probabilitas tertentu.
3. Dapat menangani **data berdimensi tinggi** dengan kinerja yang baik.
4. Membutuhkan **upaya pelatihan yang relatif lebih sedikit** dibandingkan algoritma lain.
5. Dapat digunakan untuk mengklasifikasi **data yang tidak dapat dipisahkan secara linear**.

Kekurangan Decision Tree

1. **Rentan terhadap overfitting**, terutama jika pohon terlalu dalam.
2. **Bias terhadap kelas dominan** pada dataset yang tidak seimbang, sehingga kinerjanya bisa menurun.
3. **Sensitif terhadap perubahan kecil pada data**, di mana sedikit perubahan dalam dataset dapat menghasilkan struktur pohon yang berbeda, yang bisa menjadi masalah dalam beberapa kasus.



Ensemble Learning

Ensemble Learning

Definisi Ensemble Learning

Ensemble Learning adalah teknik dalam Machine Learning yang menggabungkan beberapa model prediktif untuk membuat prediksi yang lebih akurat, stabil, dan tahan terhadap noise. Tujuannya adalah meningkatkan kinerja model dengan mengkombinasikan kekuatan beberapa model agar saling mengkompensasi kelemahan satu sama lain.

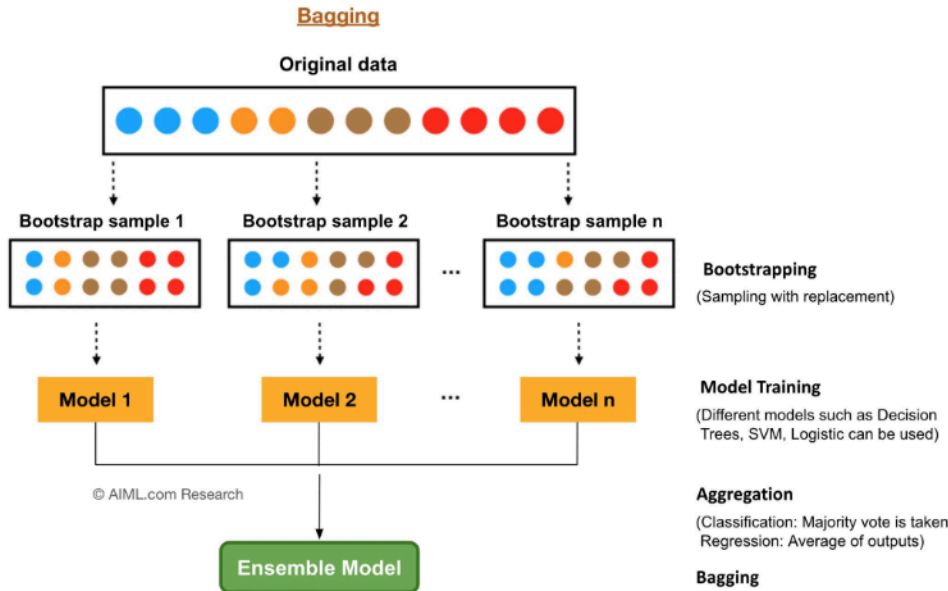
Daripada hanya bergantung pada satu model, Ensemble Learning menciptakan model gabungan yang lebih robust dan generalizable. Teknik ini sering digunakan untuk mengatasi masalah seperti overfitting ataupun underfitting.

Ada 3 Jenis Ensemble Learning yang akan kita pelajari, yaitu :

1. Bagging
2. Boosting
3. Stacking

Ensemble Learning

Bagging



- **Bagging** adalah singkatan dari **Bootstrap Aggregating**.
- Dalam bagging, beberapa model dilatih secara independen pada subset data pelatihan yang berbeda. Subset data ini diperoleh melalui teknik bootstrapping, di mana data diambil secara acak dengan kemungkinan terpilih lebih dari satu kali.
- Setelah model-model tersebut dilatih, hasil prediksinya digabungkan. Untuk **regresi**, hasil digabungkan dengan **rata-rata**, sedangkan untuk **klasifikasi**, hasil digabungkan dengan **voting mayoritas**.
- **Random Forest** adalah salah satu contoh dari Bagging (Kumpulan dari beberapa Decision Tree)

Ensemble Learning

Boosting

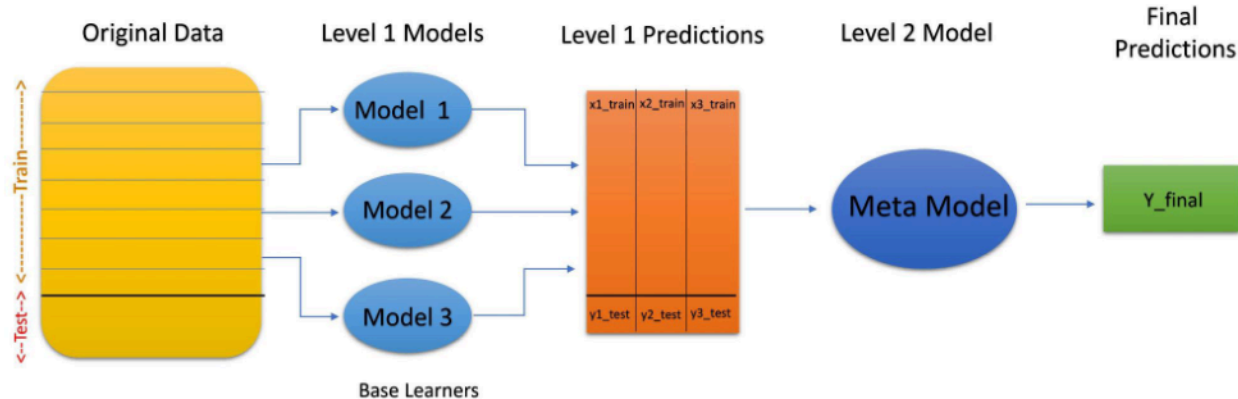


- **Boosting** adalah metode **ensemble learning** yang bertujuan meningkatkan akurasi model dengan cara **melatih model secara bertahap**, di mana setiap model berikutnya berfokus untuk memperbaiki kesalahan yang dibuat oleh model sebelumnya. Dalam Boosting, model-model yang lebih lemah (*weak learners*) digabungkan untuk menghasilkan model yang lebih kuat dan akurat.
- Data yang sulit diprediksi atau salah diklasifikasikan oleh model sebelumnya akan mendapatkan bobot yang lebih tinggi. Ini memaksa model berikutnya untuk lebih fokus pada kesalahan tersebut, meningkatkan akurasi keseluruhan.

Ensemble Learning

Stacking

Stacking adalah salah satu teknik dalam **Ensemble Learning** yang menggabungkan prediksi dari beberapa model (dengan jenis yang berbeda) untuk menghasilkan prediksi yang lebih akurat dan kuat. Teknik ini melibatkan pelatihan model-model dasar (base models) dan kemudian menggunakan model tambahan (meta-model) untuk menggabungkan prediksi dari model-model dasar tersebut.



Ensemble Learning

Stacking

Cara Kerja Stacking:

- Langkah pertama adalah melatih beberapa model dasar yang berbeda menggunakan dataset yang sama. Model dasar ini bisa terdiri dari berbagai jenis algoritma, seperti **Decision Trees**, **SVM**, **Logistic Regression**, atau **Neural Networks**. Setiap model dasar belajar dari data latih dan menghasilkan prediksi.
- Setelah model dasar dilatih, mereka digunakan untuk memprediksi output (target) pada data yang sama (atau data validasi jika ada). Prediksi dari model-model dasar ini digunakan sebagai input untuk langkah berikutnya.
- Prediksi yang dihasilkan oleh model dasar (dalam bentuk fitur) akan digunakan sebagai input untuk **meta-model**. Meta-model adalah model tingkat kedua yang dilatih untuk mempelajari cara menggabungkan prediksi dari model-model dasar.
- Setelah meta-model dilatih, prediksi akhir dibuat dengan menggunakan prediksi dari model dasar sebagai inputnya.
- Meta-model menghasilkan prediksi yang lebih baik karena dapat menggabungkan kekuatan dari berbagai model dasar dan mengoptimalkan prediksi berdasarkan pola yang ditemukan oleh model dasar.



Hyperparameter Tuning

Hyperparameter Tuning

Definisi Hyperparameter Tuning

Hyperparameter tuning adalah proses untuk menemukan kombinasi terbaik dari hyperparameter pada model machine learning yang dapat meningkatkan kinerja model pada data yang tidak terlihat (unseen data). Hyperparameter adalah parameter yang tidak dipelajari langsung selama pelatihan model, melainkan diatur sebelumnya sebelum pelatihan dimulai.

Contoh hyperparameter dalam berbagai model:

- **Model Machine Learning Traditional:** Jumlah pohon (`n_estimators`) dalam random forest, kedalaman maksimal (`max depth`) dari Decision Tree.
- **Model neural network:** Jumlah lapisan tersembunyi (`hidden layers`), jumlah neuron di setiap lapisan, dan laju pembelajaran (**learning rate**).

Ada beberapa teknik yang dapat kita gunakan untuk Hyperparameter tuning seperti Grid Search, Random Search, Bayesian Optimization, dll. Namun disini kita akan menggunakan **Grid Search** dan **Random Search**.

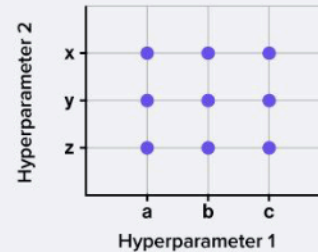
Hyperparameter Tuning

Grid Search Vs Random Search

- **Grid Search** adalah pendekatan pencarian yang melakukan eksplorasi semua kombinasi hyperparameter dalam ruang parameter yang telah ditentukan. Ini adalah metode yang sangat mendalam, tetapi dapat sangat memakan waktu karena jumlah kombinasi yang harus diuji.
- **Random Search** adalah pendekatan pencarian hyperparameter yang berbeda dengan **Grid Search**. Alih-alih mengeksplorasi semua kombinasi hyperparameter yang mungkin, **Random Search** memilih kombinasi hyperparameter secara acak dari ruang parameter yang sudah ditentukan.

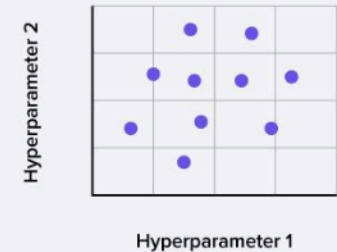
Grid Search

Hyperparameter_One = [a, b, c]
Hyperparameter_Two = [x, y, z]
Hyperparameter_X = [i, j, k]



Random Search

Hyperparameter_One = random.num (range)
Hyperparameter_Two = random.num (range)
Hyperparameter_X = random.num (range)



Sumber gambar: <https://www.sabrepc.com/blog/>



External References (Practice)

Colab Link

[Visit Here](#)



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia